

Complexité et notation *Big O*

Novembre 2025

Ce document couvre les notions de complexité des algorithmes et en particulier la notation *big O*

La notation *Big O* a pour but d'exprimer la complexité temporelle des algorithmes dans le pire scenario d'exécution (*Worst Case scenario*) et suit quelques principes de base :

- Les constantes sont ignorées
 $O(2n) = 2 \cdot O(n) = O(n)$
- Les termes dominés sont ignorés
 $O(2n^2 + 5n + 50) = O(n^2)$
- Uniquement le pire cas d'exécution est considéré

Complexités les plus fréquentes

Les complexités les plus fréquentes sont les suivantes :

Constante $O(1)$

Cela veut dire que la complexité ne dépend pas de la taille du vecteur en input.

- Les méthodes *append* et *pop* sur les listes en Python.

Logarithmique $O(\log(n))$

- L'algorithme de recherche binaire

Linéaire $O(n)$

- Les boucles *for* et *while*

Quadratique, Cubique, etc. $O(n^2)$, $O(n^3)$

- Boucles imbriquées ayant le même nombre d'itérations
- L'algorithme de tri par insertion tel que vu en cours à une complexité de $O(n^2)$

Exponentielle $O(2^n)$

- Fibonacci par récursivité comme vu en exercice

Exemples

La boucle ci-dessous va exécuter n fois (le n ici est choisi arbitrairement et représente la taille du vecteur *array*) une opération de complexité $O(1)$, la complexité totale est donc $n \cdot O(1) = O(n)$.

```
1 for number in array:
2     array.pop()
```

Dans le prochain exemple nous avons deux vecteurs, si leur taille est respectivement de n et m alors la complexité sera de $O(n \cdot m)$ et non $O(n^2)$. Cela s'explique par le fait que les deux boucles imbriquées n'itérent pas sur le même vecteur.

```

1 def intersection_size(arrayA, arrayB):
2     count = 0
3     for a in arrayA:
4         for b in arrayB:
5             if a == b:
6                 count += 1
7     return count

```

Les deux fonctions ci-dessous ont la même complexité de $O(n)$

```

1 def find_extrem_1(array):
2     mini , maxi = (array[0], )*2
3     for numbers in array:
4         if numbers < mini:
5             mini = numbers
6     for numbers in array:
7         if numbers > maxi:
8             maxi = numbers
9     return mini , maxi
10
11 def find_extrem_2(array):
12     mini , maxi = (array[0], )*2
13     for numbers in array:
14         if numbers < mini:
15             mini = numbers
16         elif numbers > maxi:
17             maxi = numbers
18     return mini , maxi

```